CSI Data extraction for gait research
Frist of all I have the following computer configuration:
Processor: Intel i7 10th gen
NIC card: Intel 5300 ngw
SSD: 256GB
I want a high quality and future proof Q1 level journal to do on gait recognition using wifi csi data. Give me the stepwise gude and pipeline from computer environment setup to full research completion on it. It should include:
Step by step process to setup environment for Intel 5300 nic for data collection
All ubuntu commands for setup of the environment along with cross-checking commands at every step if the environment is in right state till that command.
I need a gui interface so, that I can control all the parameters during data collection
The collected data should be in the best format (i.e .hdf, csv or other which will be the best according to your suggestion) so that it can be used for future research purpose.
For every code/command , give explanation why did we do.
Now, give me first to last guide

I'm mapping this into two parts: a stable **data-collection stack** for the Intel 5300, and a **future-proof research stack** for storage, preprocessing, modeling, and publication. I'll verify the 5300 setup details from primary sources first, then give you the end-to-end pipeline with commands, checkpoints, and the reasoning behind each step.

A couple of important facts are already clear. The official 5300 CSI tool is **old and kernel-sensitive**—its authors say it is expected to work with upstream kernels roughly **3.2 to 4.2**—and they also note that **monitor-mode injection** is usually more reliable than normal AP/client mode for experiments. dhalperi.github.io +1

So the future-proof way to do this is not "run everything on one modern OS." It is: keep a **dedicated legacy collection machine** for CSI capture, and store raw data in a **modern**

research format for later processing and modeling on a newer Ubuntu/Python stack.

I've got the collection side pinned down now. One key design choice for your project is this:

- For **the actual Intel 5300 setup**, the safest path is still the classic CSI-tool workflow.
- For a **GUI and future-proof research workflow**, the smarter architecture is to keep 5300 for raw capture but convert everything immediately into a modern dataset layout and drive collection with your own thin GUI wrapper.
- If you later want a more modern CSI platform, current tools like **PicoScenes** and **FeitCSI** are much more forward-looking; PicoScenes explicitly supports **IWL5300**, and FeitCSI explicitly offers a **GUI** but targets newer Intel NICs like **AX200/AX210**, not the 5300. ps.zpj.io +2

Thought for 8m 38s ›

Yes — but the **right way** to make this both publishable and future-proof is to separate the project into two worlds:

1. a **legacy collection node** for the Intel 5300, because the original CSI stack is old and kernel-sensitive, and
2. a **modern research node** for preprocessing, storage, training, evaluation, and paper writing.

Think of it like using an old but trusted scientific instrument: you do **measurement** on the old instrument, but you do **analysis and publication** on a modern lab workstation. The original Intel 5300 CSI tool is officially documented for upstream Linux kernels roughly **3.2 to 4.2**, while a later community fork reports testing on **Ubuntu 16.04/18.04 with kernel 4.15**. The official tool records CSI as binary traces via `log_to_file`, and the FAQ also says the setup is usually most effective in **monitor-mode injection**, not normal AP/client mode. dhalperi.github.io +3

The architecture I recommend

Use your Intel 5300 machine as a **dedicated collector**. For that machine, use the CSI tool path below. Immediately after collection, convert every raw trace into **HDF5** for long-term storage, and optionally export derived tabular features to **Parquet**. HDF5 is a strong fit because it supports hierarchical groups, datasets, attributes, extendable datasets, chunking, and compression; Parquet is best for compressed, column-oriented feature tables used in analytics pipelines. Keep the original `csi.dat` too, because raw preservation matters for reproducibility. The HDF Group … +4

A realistic journal-grade research target

A weak paper in this area says, “I got high accuracy in one room on known people.” A stronger Q1-style paper says, “I built a reproducible WiFi-CSI gait identification pipeline that still works across **rooms, days, clothing states, walking speeds, and unknown users**.” That matters because recent work still highlights problems with **increasing user counts**, **insufficient samples**, **cross-domain generalization**, and **open-set deployment**. ScienceDirect +3

The best paper idea for you is:

Cross-environment, open-set WiFi-CSI gait identification using Intel 5300, with a public HDF5 dataset, strong baselines, and a modern domain-generalized model.

That framing fits current Q1 venues much better than “just another in-domain classifier.” For biometrics/security, strong current Q1 fits include **IEEE T-BIOM** and **IEEE TIFS**; for healthcare-oriented abnormal gait work, **IEEE JBHI** is a strong Q1 fit; for broader AI/pattern-recognition framing, **Pattern Recognition** and **IEEE Internet of Things Journal** are also current Q1 venues. Quartiles can change by year and category, so always re-check before submission. Scimago Journal… +4

Part 1 — Collection machine strategy

Step 0: Decide the OS path before touching drivers

There are two practical paths.

Path A — safest documented path

Use the **official Linux 802.11n CSI Tool** on a collector OS whose kernel is in the documented working window, roughly **3.2–4.2**. This is the path with the clearest official instructions. dhalperi.github.io

Path B — practical compromise

Use the **spanev** fork if you want a somewhat newer collector environment; that fork says it has been tested on **Ubuntu 16.04/18.04 with kernel 4.15**. GitHub

For a beginner who wants the highest chance of success, I would do this:

- **Collector machine:** Ubuntu aligned with the official CSI-tool path.

- **Research machine:** modern Ubuntu 22.04/24.04 for Python, HDF5, PyTorch, and experiments.

If your current 10th-gen machine is awkward with very old Ubuntu, use a **second SSD** or a **separate older machine** as the collector. That is the cleanest setup.

Part 2 — Intel 5300 environment setup, first to last

Below I'll give the **official-style collector flow**, with a checkpoint after every stage.

Step 1: Verify hardware, kernel, interface, and Secure Boot

Run:

```bash
uname -r
lspci | grep -i -E 'network|wireless'
sudo lshw -C network
```

Why we do this

- `uname -r` tells you your current kernel version, which is critical because the original CSI tool is version-sensitive.
- `lspci` and `lshw` confirm that the Intel 5300 is physically detected.
- `iw dev` shows the wireless interface name, which may be `wlan0`, `wlp...`, or something else.
- `mokutil --sb-state` checks Secure Boot. If module signature enforcement is active, custom wireless modules can fail to load unless signed. The official CSI-tool page notes unsigned-module warnings are only harmless when signature verification is not enforced; Ubuntu's Secure Boot docs explain how third-party module signing works. dhalperi.github.io +3

Cross-check

You want to see:

- the Intel 5300 listed by PCI,
- a wireless interface visible,

- and ideally Secure Boot either disabled, or you already know how you will handle signing.

Step 2: Install required packages

Run:

```bash
```

Why we do this

The official installation instructions require the build tools, kernel headers, Git, and command-line wireless tools. `gcc` and `make` compile the modified driver, `linux-headers-$(uname -r)` gives the exact headers for your running kernel, `git-core` pulls the source tree, and `iw` lets you control the interface directly. dhalperi.github.io

Cross-check

```bash
gcc --version
make --version
```

You should see:

- GCC and Make versions,
- a valid `build` directory for your kernel,
- and at least one wireless device.

Step 3: Stop NetworkManager from fighting your experiments

If your interface is `wlan0`, run:

```bash
```

```
sudo restart network-manager
```

On newer systemd-based Ubuntu, if `restart` does not work, use:

Bash

Why we do this

The official guide recommends using CLI control (`iw` , `iproute2`) and disabling NetworkManager's normal control of the experiment interface. Otherwise, your GUI applet may keep reconfiguring the card while you are trying to put it into monitor mode or fixed experimental states. dhalperi.github.io

Cross-check

Bash

Your WiFi interface should not be actively managed in a way that interferes with manual control.

Step 4: Optional blacklist for development convenience

Run:

Bash

Why we do this

The official guide suggests blacklisting the stock modules if you want tighter control over when the wireless driver loads, especially while developing or troubleshooting modified drivers. This is optional, but it prevents the stock path from reappearing unexpectedly at boot. dhalperi.github.io

Cross-check

Bash

You should see the two blacklist lines.

Step 5: Clone the CSI-tool driver source and select the right tag

Run:

```bash
export CSITOOL_KERNEL_TAG=csitool-$(uname -r | cut -d . -f 1-2)
git clone https://github.com/dhalperi/linux-80211n-csitool.git
```

Bash

Why we do this

The official instructions map the running kernel's major.minor version to the corresponding CSI-tool tag. This is how you align the modified driver code with the kernel family you are actually running. dhalperi.github.io

Cross-check

```bash
echo $CSITOOL_KERNEL_TAG
git tag | grep csitool
```

Bash

If the `git checkout` fails, your kernel is probably outside the supported range for the official path.

Step 6: Optional Ubuntu kernel merge step

Only do this if the plain driver build fails or you specifically need Ubuntu-kernel compatibility.

Run:

```bash
cat /proc/version_signature
. /etc/lsb-release
git remote add ubuntu git://kernel.ubuntu.com/ubuntu/ubuntu-${DISTRIB_CODE
# Set this manually after checking your exact Ubuntu kernel tag:
```

Bash

Why we do this

The official instructions say you can merge the CSI-tool modifications into the distribution-provided Ubuntu kernel tree for better compatibility. This is not always necessary, but it is the official escape hatch when the simple path is not enough.

dhalperi.github.io +1

Cross-check

Bash

You should see the Ubuntu remote and recent merged commits.

Step 7: Build the modified Intel wireless driver

Run:

Bash

Why we do this

This compiles only the modified `iwlwifi` module against your current kernel headers, instead of rebuilding the entire kernel. That is exactly what the official installation guide prescribes. dhalperi.github.io

Cross-check

```bash
```

A successful compile should produce object/module artifacts and exit cleanly.

Step 8: Install the modified driver

Run:

```bash
sudo make -C /lib/modules/$(uname -r)/build M=$(pwd)/drivers/net/wireless/
sudo depmod
```

Why we do this

The first command installs the new module into the kernel's `updates` path, and `depmod` rebuilds dependency metadata so the kernel can find and load it cleanly. The official instructions use this exact pattern. dhalperi.github.io

Cross-check

```bash
```

You want to see the installed updated module and valid module metadata.

Step 9: Clone the supplementary repository and replace the firmware

Run:

```bash
git clone https://github.com/dhalperi/linux-80211n-csitool-supplementary.g

for file in /lib/firmware/iwlwifi-5000-*.ucode; do
    sudo mv "$file" "$file.orig"
done

sudo cp linux-80211n-csitool-supplementary/firmware/iwlwifi-5000-2.ucode.s
```

Bash

Why we do this

The Intel 5300 CSI capability depends on the **modified firmware** shipped with the supplementary repository. Moving the original firmware aside preserves rollback ability, while the symlink points the driver to the CSI-enabled firmware image. dhalperi.github.io +1

Cross-check

Bash

You should see:

- the `.sigcomm2010` firmware file,
- and a symlink named `iwlwifi-5000-2.ucode` pointing to it.

Step 10: Build the userspace logger

Run:

Bash

Why we do this

The official flow uses `log_to_file`, a userspace tool that receives CSI notifications from the driver and writes them into a binary file such as `csi.dat`. dhalperi.github.io +1

Cross-check

Bash

If the binary exists, the logger built correctly.

Step 11: Unload stock modules and reload with CSI logging enabled

Run:

Bash

If that says the module is in use, run:

Bash

Then load the CSI-enabled module:

Bash

Why we do this

You need to unload any currently active wireless stack and reload `iwlwifi` with the `connector_log` flag so CSI notifications are exposed to userspace. The official FAQ also notes that you can change the logging mask later if you want more than beamforming notifications. dhalperi.github.io +1

Cross-check

Bash

You should see the module loaded and the firmware starting without fatal errors.

Step 12: First smoke test in normal connected mode

Connect to an **802.11n HT** access point.

Then start logging:

```bash
```

In another terminal, generate traffic:

```bash
```

Why we do this

This is the official first test. The logger writes CSI for each received packet that meets the CSI conditions. The FAQ says CSI is only measured on **received 802.11n HT packets** with compatible configuration; the first packets are often legacy-rate packets and may not yield CSI. dhalperi.github.io +1

Cross-check

```bash
```

And in a second terminal:

```bash
```

The file size should grow while traffic is flowing.

Part 3 — What most beginners miss

The encrypted AP trap

The official FAQ says the modified firmware did not have enough room for both CSI paths and normal WEP/WPA/WPA2 encryption paths, so encrypted APs are a known problem for the original tool. For serious collection, either use an **open test network** for the smoke test or move to **monitor-mode injection** for stable experiments. dhalperi.github.io

The AP/client reliability trap

The official FAQ is blunt: the authors found the tool most effective with **injection scripts in monitor mode**; client mode, AP mode, and ad-hoc mode were not as reliable. So do not stop at the ping-based smoke test. Use that only to validate the install. For real gait research, move to monitor mode. dhalperi.github.io

---

Part 4 — Research-grade collection mode

Step 13: Put the interface into monitor mode

The supplementary repository includes a helper script whose indexed contents show it does this sequence:

- unloads `iwlwifi/mac80211/cfg80211`,
- reloads `iwlwifi connector_log=0x1`,
- repeatedly tries `iwconfig wlan0 mode monitor`,
- sets the channel with `iw`,
- then brings the interface up. GitHub

So from the supplementary injection folder, run:

Bash

If your interface is not `wlan0`, edit the script first and replace `wlan0` with your actual interface name.

Why we do this

Monitor mode gives you tighter control and aligns with the collection mode the original authors recommend for stable CSI experiments. `6 HT20` means channel 6, 20 MHz bandwidth. Start with HT20 before trying HT40. dhalperi.github.io +1

Cross-check

```
iw dev
```

Bash

You want to see `type monitor`.

Step 14: Fix the monitor transmit rate

Run:

Bash

Why we do this

The official FAQ shows that in monitor-mode injection you can write a rate value into debugfs to select the monitor transmit rate. This matters because CSI exists only for compatible HT packets, and fixed-rate experiments are far easier to reproduce than "whatever the rate-control algorithm decided today." dhalperi.github.io

Cross-check

Bash

It should print the configured value.

Step 15: Start logging in research mode

Run:

Bash

Why we do this

This is the same logger as before, but now your collection environment is much more controlled. Use one raw file per session. Never overwrite. Raw-first is the right habit.

dhalperi.github.io +1

Cross-check

Bash

You want the file to grow steadily.

Part 5 — The GUI you asked for

The original Intel 5300 path is fundamentally **CLI-first**: driver build, module reload, `log_to_file`, helper scripts. If you want a real GUI while staying with the 5300, the clean solution is a **thin GUI wrapper** around those commands. That wrapper should not replace the low-level toolchain; it should simply orchestrate it. The original tool gives you CLI logging; modern tools like FeitCSI explicitly provide a GUI but target newer Intel NICs such as AX200/AX210, while PicoScenes is a more modern sensing platform that supports IWL5300 and offers live CSI plotting and richer controls. dhalperi.github.io +2

Step 16: Create a modern Python environment for the GUI and conversion tools

On your modern research environment, run:

```bash
sudo apt update
sudo apt install python3 python3-venv python3-pip hdf5-tools
python3 -m venv ~/venvs/csi-research
source ~/venvs/csi-research/bin/activate
```

Bash

Why we do this

- `h5py` lets Python write HDF5, which is excellent for large numerical, structured CSI archives.

- `pyarrow` gives you Parquet support for fast feature tables.
- `PyQt6` and `pyqtgraph` are good for a lightweight live GUI.
- `CSIKit` already supports parsing Intel IWL5300 and can also generate CSV/JSON when needed. docs.h5py.org +3

Cross-check

```bash
python - <<'PY'
import numpy, scipy, pandas, h5py, pyarrow, matplotlib, sklearn, yaml
```

And:

```bash
python - <<'PY'
import CSIKit
```

What the GUI should control

Your GUI should expose only the knobs that matter:

- interface name,
- channel,
- bandwidth (`HT20` first),
- session ID,
- subject ID,
- walking direction,
- room ID,
- LOS/NLOS,
- number of trials,
- output folder,
- start/stop logging,
- live amplitude plot for a few selected antenna/subcarrier streams.

In simple words: the GUI is your **cockpit**, but the original CSI stack is still the **engine**.

A good design is:

- one YAML profile per session,
- GUI launches `setup_monitor_csi.sh`,
- GUI launches `log_to_file`,
- GUI timestamps start/stop events,
- GUI writes `session.json`,
- then automatically calls your conversion script to produce `session.h5`.

Part 6 — Best data format for future research

My recommendation

Use **three layers**:

1. **Raw:** `sessionXX_raw.csi.dat`
2. **Canonical archive:** `sessionXX.h5`
3. **Derived features:** `segments.parquet`

That is the best long-term structure.

Why not CSV for raw CSI?

CSV is fine for tiny demos, but CSI is high-dimensional, packet-level, and often large. CSV becomes bloated, slow, and loses the natural hierarchy of packets, antennas, subcarriers, timestamps, labels, and environment metadata.

Why HDF5 is best for the canonical file

HDF5 is designed exactly for this kind of hierarchical scientific data: groups, datasets, attributes, extendable arrays, chunking, and compression. That makes it ideal for packet-level CSI plus metadata in one portable scientific file. The HDF Group … +3

Why Parquet is best for features

Parquet is column-oriented and optimized for efficient storage and retrieval of tabular data. That makes it excellent for:

- segment-level features,
- spectrogram statistics,
- Doppler summaries,
- train/val/test split tables,

- ablation-ready metadata tables. Parquet +1

Recommended on-disk layout

```
gait_csi_project/
  raw/
    2026-03-19/
      subj_001/
        sess_001/
          session_raw.csi.dat
          session.json
  canonical/
    2026-03-19/
      subj_001/
        sess_001.h5
  features/
    segments.parquet
  labels/
    subject_registry.csv
    split_definition.csv
  code/
  notebooks/
  experiments/
  papers/
```

Recommended HDF5 schema

```
/session
  /csi
    data              # shape: [packets, rx, tx, subcarrier, complex]
    amplitude         # optional cached amplitude
    phase             # optional cached phase
  /packet
    timestamp_us
    rssi_a
    rssi_b
    rssi_c
    noise
    rate
    antenna_sel
  /labels
    subject_id
    trial_id
    path_id
    direction
    room_id
```

```
      date_id
      los_flag
      footwear
      clothing_state
    /meta
      tool_name
      tool_version
      kernel_version
```



```
      bandwidth
      tx_rx_distance_m
      notes
```

That schema is strong because it preserves both **signal** and **experiment context**.

Part 7 — The full research pipeline, from raw data to paper

Phase 1: Problem definition

Pick one of these and commit to it early:

Option 1: Closed-set gait identification

Known people only. Easiest. Good for starting, but weak novelty alone.

Option 2: Cross-environment identification

Train in one room/day, test in another. Much stronger.

Option 3: Open-set identification

Known users plus unknown-user rejection. This is highly practical and currently much more publication-worthy. Recent work explicitly frames open-set gait recognition as a critical real-world problem. scilit.com

Best choice: combine **cross-environment + open-set**.

Phase 2: Experimental protocol

Design your data collection like a real biometrics study.

Subjects

Target:

- **20–40 people** minimum for a solid first paper,
- balanced gender if possible,
- repeated sessions on **3–5 different days**.

Environments

Collect in:

- at least **2 rooms**,
- ideally **LOS and NLOS** variants,
- more than one walking path.

Variability

Deliberately vary:

- walking speed,
- shoes/slippers,
- carrying small objects,
- clothing,
- morning/evening sessions.

Why this matters

Recent WiFi-CSI sensing literature keeps stressing domain dependence and transferability problems. If your dataset does not include realistic variability, your paper will likely become another “works only in my lab” result. Springer +1

Phase 3: Segmentation and preprocessing

Your preprocessing pipeline should be fixed and documented:

1. Parse raw `csi.dat`.
2. Remove corrupted packets.
3. Choose antenna/subcarrier streams.
4. Build amplitude and calibrated phase representations.
5. Denoise with light filtering.
6. Detect walking windows.
7. Normalize per session and per subject.
8. Generate segment-level training samples.

Big-picture intuition

Raw CSI is like a shaky video. Preprocessing is how you stabilize the camera before asking the model to recognize a person.

Strong baseline representations

Use all three:

- amplitude only,
- amplitude + phase,
- time-frequency representation (spectrogram/CWT/STFT),
- optionally Doppler-like features.

Recent papers continue to improve by extracting richer multi-view or hybrid temporal features rather than relying on a single view. Springer +2

Phase 4: Modeling strategy

Start with three baseline families.

Baseline A: classical ML

SVM / Random Forest on handcrafted features.

Why: gives you a sanity-check floor.

Baseline B: CNN + temporal model

1D-CNN or 2D-CNN on spectrograms, followed by BiGRU/LSTM.

Why: many WiFi gait papers still use this family effectively. Springer

Baseline C: Transformer-based model

Temporal transformer or patch-based spectrogram transformer.

Why: newer work is moving in this direction, and it is more defensible in a modern Q1 submission. pure.hud.ac.uk

My recommended final model

Build a model with:

- packet/subcarrier encoder,
- temporal transformer,

- domain-adversarial head,
- unknown-user rejection head,
- confidence calibration.

That turns your paper from “recognition only” into “recognition under deployment conditions.”

Phase 5: Evaluation protocol

Do **not** report only accuracy.

Report at least:

- accuracy,
- macro-F1,
- confusion matrix,
- ROC/AUROC for unknown-user rejection,
- per-room and per-day performance,
- inference time,
- model size,
- ablation study.

Also report these split settings:

- in-domain,
- cross-day,
- cross-room,
- cross-day + cross-room,
- open-set.

That is the difference between a classroom project and a journal paper.

Phase 6: Benchmarking

Benchmark against public resources too. Widar3 explicitly notes gait-related data availability, and SenseFi provides a benchmark/library intended to validate learning-based WiFi sensing methods on multiple datasets and platforms. Even if your final contribution is your own dataset, comparing your model against known public benchmarks improves credibility a lot. tns.thss.tsinghu… +1

Phase 7: Reproducibility package

For a strong journal submission, release:

- raw example traces,
- full HDF5 schema,
- conversion code,
- preprocessing code,
- model code,
- exact splits,
- environment YAML/requirements,
- data collection geometry diagrams.

A WiFi-CSI paper becomes much stronger when others can actually rebuild the pipeline.

Part 8 — My exact publication roadmap for you

Paper concept

Open-set cross-environment WiFi-CSI gait identification using Intel 5300 with hierarchical HDF5 dataset release and uncertainty-aware domain generalization

Claimed contributions

1. A reproducible Intel 5300 collection stack.
2. A future-proof HDF5 CSI archive format.
3. A new multi-day, multi-room gait dataset.
4. A cross-environment + open-set evaluation protocol.
5. A stronger temporal model than plain CNN/LSTM baselines.
6. Full ablations and public code.

Best journal match

- **T-BIOM** if your framing is biometrics/identity science. Scimago Journal… +1
- **TIFS** if your framing is authentication/security. Scimago Journal…
- **JBHI** if you pivot toward abnormal/clinical gait. Scimago Journal…
- **Pattern Recognition** if the main story is model/method rather than sensing hardware. Scimago Journal…
- **IEEE IoT Journal** if the story is deployable sensing systems. Scimago Journal…

Part 9 — What I would do in your shoes

1. Finish the official Intel 5300 install until `csi.dat` grows during the ping test.
2. Move immediately to **monitor-mode collection**.
3. Store every session as raw `csi.dat` + `session.json` + converted `session.h5`.
4. Build a small PyQt GUI to launch collection profiles.
5. Collect a **multi-day, multi-room** dataset from the start.
6. Train baselines first, then add domain generalization and open-set recognition.
7. Aim the paper at **T-BIOM** or **TIFS**.

The single most important thing here is this: **do not spend 6 months collecting a small, single-room dataset.** That is the most common way this kind of project becomes unpublishable.

The next best move is to execute only up to **Step 12** first. Once the install is validated and `csi.dat` is growing, then build the GUI and the HDF5 conversion layer on top of a working base.

Sources

which version of ubuntu will have to be used for this. will your setup guideline work for the latest ubuntu

I'm checking two things before answering directly: the CSI tool's real kernel support window, and what "latest Ubuntu" means today from Ubuntu's own sources. Then I'll give you the practical recommendation, not just the theoretical one.

Thought for 47s ›

No — **not for the Intel 5300 CSI setup I gave you**.

The original Intel 5300 CSI Tool says its installation instructions are expected to work on Linux systems using upstream kernel versions roughly **3.2 to 4.2**, with examples like **Ubuntu 12.04** and **Ubuntu 14.04.4**. A later community fork for "newer kernels" says it has been tested on **Ubuntu 16.04 and Ubuntu 18.04 with kernel 4.15**, and its release is specifically labeled for **kernel v4.15**. dhalperi.github.io +2

So for **your Intel 5300 NIC**, my practical recommendation is:

- **Best practical choice: Ubuntu 18.04.0 with kernel 4.15**, using the newer community fork. GitHub +2
- **Most conservative / closest to the original docs: Ubuntu 14.04.4** or another setup in the original kernel window. dhalperi.github.io
- **Do not use latest Ubuntu for the collector machine.** The current Ubuntu releases page shows **Ubuntu 24.04.4** as a released version, while **26.04 LTS** is still shown as a **daily build**, not a finished release. Since 24.04.4 is far newer than the CSI tool's documented kernel support, it is not a reliable target for the Intel 5300 CSI driver path. Ubuntu Releases +4

Big picture: the Intel 5300 CSI stack is like an **old lab instrument with old drivers**. Your laptop hardware may be modern, but the CSI software layer is not. That is why I suggested a **legacy Ubuntu just for data collection**, and a **modern Ubuntu for analysis and model training**. That split is the safest and most future-proof approach.

dhalperi.github.io +1

My exact answer to your question is:

- **Which Ubuntu should you use?**
  Use **Ubuntu 18.04.0, kernel 4.15** for the Intel 5300 collection machine.
- **Will my setup guideline work on the latest Ubuntu?**
  No, not reliably. For **Ubuntu 24.04.4**, I would not trust the Intel 5300 CSI driver setup to work cleanly, because the documented and community-supported kernel ranges are much older. Ubuntu Releases +2

There is one extra practical warning: your **10th-gen i7 laptop** is much newer than Ubuntu 14.04/18.04-era kernels, so the cleanest real-world setup is often:

- one partition / SSD / separate machine with **Ubuntu 18.04.0 + Intel 5300** for collection,
- and another modern Ubuntu install for analysis.

In other words: **latest Ubuntu for research code, old Ubuntu for CSI capture**.

In the next message, I can give you the **exact Ubuntu 18.04.0 installation plan**, including whether to use **18.04.0 specifically vs 18.04.6**, how to pin the kernel to **4.15**, and the full command-by-command collector setup with checkpoints.

Sources

yes, go for it

I'm locking down the exact collector OS choice and the one trap that matters most here: some 18.04 installers drift onto newer HWE kernels, which can quietly break the 5300 path. Then I'll give you the precise install flow and the checkpoints after each stage.

One more practical issue I'm checking because it affects your exact laptop: Ubuntu 18.04 with kernel 4.15 is good for the 5300 tool, but your **10th-gen Intel platform is much newer** than that kernel. If that mismatch is real, I'll tell you the safest workaround instead of pretending the collector OS will be comfortable on your main machine.

Thought for 4m 59s ›

Good. Use **Ubuntu 18.04.0 Desktop AMD64 exactly** for the **collector machine**. The reason is simple: Ubuntu 18.04 ships with a **4.15** kernel, Ubuntu's HWE mechanism can move an LTS release to newer kernels, and the community-maintained Intel 5300 CSI fork explicitly says the commands are for **kernel 4.15** and were tested on **Ubuntu 18.04 with kernel 4.15**. Also, Ubuntu 18.04 is now **out of standard support**, and the ISO is kept on Ubuntu's **old-releases** archive. old-releases.ubu… +4

So the exact answer is:

- **Collector OS:** `ubuntu-18.04-desktop-amd64.iso`
- **Kernel target:** any **4.15.x** kernel
- **Do not use latest Ubuntu for Intel 5300 CSI collection**
- **Do not let this machine drift to HWE 5.x kernels**
- **Use Ubuntu 18.04 Desktop**, not Server, because you asked for a GUI. old-releases.ubu… +3

Think of this machine as a **dedicated lab instrument**, not your everyday laptop OS. The collector can be old and stable; your preprocessing, HDF5 conversion, modeling, and paper writing can happen later on a modern Ubuntu system. Because 18.04 is out of standard support, keep the collector on a controlled lab network or mostly offline except when you need packages. Ubuntu +1

Exact Ubuntu 18.04.0 collector installation plan

1) Download the exact ISO

On another Linux machine, run:

```bash
mkdir -p ~/Downloads/ubuntu1804-csi
cd ~/Downloads/ubuntu1804-csi

wget http://old-releases.ubuntu.com/releases/18.04.0/ubuntu-18.04-desktop-
wget http://old-releases.ubuntu.com/releases/18.04.0/SHA256SUMS
wget http://old-releases.ubuntu.com/releases/18.04.0/SHA256SUMS.gpg
```

Why

You want the **original 18.04.0 desktop image**, not a later point release. The old-releases archive shows that exact ISO and also provides `SHA256SUMS`. old-releases.ubu...

Cross-check

```bash
ls -lh
grep 'ubuntu-18.04-desktop-amd64.iso' SHA256SUMS
sha256sum ubuntu-18.04-desktop-amd64.iso
```

Match the checksum output with the line from `SHA256SUMS`.

---

2) Create the bootable USB

Insert an empty USB drive and identify it:

```bash
lsblk
```

Suppose the USB is `/dev/sdb`. Then:

```bash
sudo umount /dev/sdb*
sudo dd if=ubuntu-18.04-desktop-amd64.iso of=/dev/sdb bs=4M status=progres
```

```
sync
```

Why

`dd` writes the ISO bit-for-bit to the USB so the machine can boot from it.

Cross-check

```bash
lsblk
```

You should see the USB still present, and the `dd` command should end without an error.

---

3) BIOS / UEFI settings before install

Before booting from the USB:

- Prefer a **dedicated SSD or partition** for this collector OS.
- **Disable Secure Boot** if possible.
- Boot in normal UEFI mode.

Why

The CSI driver path uses a modified wireless module. The CSI tool docs note that module-signing warnings are only harmless when the kernel is **not enforcing module signature verification**. Disabling Secure Boot avoids extra pain here. Dhalperi

---

4) Install Ubuntu 18.04 Desktop

Boot from the USB and install Ubuntu.

Recommended installation choices

- Choose **Ubuntu Desktop**
- A normal install is fine
- Since this is a collector node, you can keep the install minimal and clean
- If you want to avoid surprises, do **not** bother with extra third-party packages during install

Why

You asked for a GUI, and Ubuntu 18.04 Desktop gives you that while keeping the kernel in the right family for the Intel 5300 CSI fork. Ubuntu Wiki +1

---

5) First boot: verify that the base system is correct

Run:

```bash
lsb_release -a
uname -r
uname -r | cut -d. -f1-2
lspci | grep -i -E 'network|wireless'
iw dev
mokutil --sb-state 2>/dev/null || echo "mokutil not available"
```

What you want to see

- Ubuntu release: **18.04**
- Kernel major.minor: **4.15**
- Intel 5300 visible in PCI output
- a wireless interface visible in `iw dev`

Why

The spanev Intel 5300 CSI fork explicitly says the kernel **should be 4.15**; otherwise the commands below won't work. GitHub

Stop rule

If `uname -r | cut -d. -f1-2` is **not** `4.15`, stop here.

---

6) Point APT to the archived Ubuntu 18.04 repositories

Because 18.04 is out of standard support, the normal mirrors may not behave the way a current release does. Repoint to the archived repository:

```bash
sudo cp /etc/apt/sources.list /etc/apt/sources.list.bak

sudo sed -i 's|http://\([a-zA-Z0-9.-]*\.\)\?archive.ubuntu.com/ubuntu|http
```

```
sudo sed -i 's|http://security.ubuntu.com/ubuntu|http://old-releases.ubunt

grep -v '^#' /etc/apt/sources.list
sudo apt update
```

Why

Ubuntu says 18.04 is out of standard support, and the old-releases archive hosts its release media. Using the archived repository path keeps package installation consistent with an old collector OS. Ubuntu +1

Cross-check

Bash

```
apt-cache policy | head -n 20
```

You should see package sources resolving cleanly.

---

7) Install the build and WiFi control tools

Run:

Bash

```
sudo apt install build-essential linux-headers-$(uname -r) git-core iw net
```

Why

The original CSI tool instructions require the build tools, kernel headers, Git, and `iw`. The 4.15 community fork uses the same basic dependency pattern. `build-essential` gives you the compiler and `make`; `linux-headers-$(uname -r)` lets you build the modified driver against the exact running kernel. Dhalperi +1

Cross-check

Bash

```
gcc --version
make --version
```

```
ls -ld /lib/modules/$(uname -r)/build
iw dev
```

You should see a valid GCC, Make, kernel build directory, and a wireless interface.

---

8) Make NetworkManager stop interfering with the experiment interface

First identify the interface name:

```bash
iw dev
```

Suppose it is `wlp3s0`. Then run:

```bash
export IFACE=wlp3s0
echo "iface ${IFACE} inet manual" | sudo tee -a /etc/network/interfaces
sudo systemctl restart NetworkManager
nmcli dev status
```

Why

The original CSI tool docs say that if you want reliable CLI control via `iw` and `iproute2`, you should stop NetworkManager from actively managing that interface. Otherwise the desktop network applet may fight your monitor-mode or manual configuration. Dhalperi +1

Cross-check

`nmcli dev status` should no longer show that interface being managed in the usual way.

---

9) Build the Intel 5300 CSI-enabled driver for Ubuntu 18.04 / kernel 4.15

Run:

```bash
```

```
cd ~
git clone https://github.com/spanev/linux-80211n-csitool.git
cd linux-80211n-csitool

export CSITOOL_KERNEL_TAG=csitool-$(uname -r | cut -d . -f 1-2)
echo $CSITOOL_KERNEL_TAG
git checkout ${CSITOOL_KERNEL_TAG}
```

Why

For Ubuntu 18.04 with kernel 4.15, you should use the **spanev fork**, not the original upstream-only path. That fork explicitly targets newer kernels and says Ubuntu 18.04 with kernel 4.15 is supported. GitHub +1

Cross-check

Bash

```
git status
git branch --show-current
```

Then build:

Bash

```
make -j"$(nproc)" -C /lib/modules/$(uname -r)/build M=$(pwd)/drivers/net/w
```

Why

This compiles only the modified `iwlwifi` module against your current kernel, which is exactly the right scope for this CSI setup. GitHub

Cross-check

Bash

```
echo $?
find drivers/net/wireless/intel/iwlwifi -name '*.ko' -o -name '*.o' | head
```

You want exit code `0` and module/object files present.

Now install it:

```bash
sudo make -C /lib/modules/$(uname -r)/build M=$(pwd)/drivers/net/wireless/
sudo depmod
cd ~
```

Why

This copies the modified module into the kernel's updates area and refreshes dependency metadata so the system can load it. The repo notes that signing-key warnings can be ignored unless the kernel is enforcing module signature verification.

GitHub +1

Cross-check

```bash
find /lib/modules/$(uname -r) -path '*updates*iwlwifi*' -type f
modinfo iwlwifi | head
```

---

10) Install the modified Intel 5300 firmware and build the logger

Run:

```bash
cd ~
git clone https://github.com/dhalperi/linux-80211n-csitool-supplementary.g

for file in /lib/firmware/iwlwifi-5000-*.ucode; do
    sudo mv "$file" "$file.orig"
done

sudo cp linux-80211n-csitool-supplementary/firmware/iwlwifi-5000-2.ucode.s
sudo ln -sf iwlwifi-5000-2.ucode.sigcomm2010 /lib/firmware/iwlwifi-5000-2.

make -C linux-80211n-csitool-supplementary/netlink
```

Why

The Intel 5300 CSI path needs the **modified firmware** from the supplementary repository, and `log_to_file` is the userspace tool that records CSI into a raw binary trace file. Dhalperi

Cross-check

```bash
ls -l /lib/firmware/iwlwifi-5000-2.ucode*
ls -l linux-80211n-csitool-supplementary/netlink/log_to_file
```

You should see:

- the `.sigcomm2010` firmware file,
- the symlink `iwlwifi-5000-2.ucode`,
- and the `log_to_file` binary.

---

11) Reload the driver with CSI logging enabled

Run:

```bash
sudo ip link set ${IFACE} down

sudo modprobe -r iwldvm iwlwifi mac80211 || sudo modprobe -r iwlwifi mac80
sudo modprobe iwlwifi connector_log=0x1
```

Why

The original CSI tool instructions say to unload the current stack and reload `iwlwifi` with `connector_log=0x1`, which enables beamforming/CSI logging to userspace.

Dhalperi +1

Cross-check

```bash
lsmod | grep -E 'iwlwifi|iwldvm|mac80211'
dmesg | tail -n 50
```

You want the driver to load cleanly and the firmware to initialize without fatal errors.

12) First smoke test: use an open 802.11n test network

Start the logger:

```bash
cd ~/linux-80211n-csitool-supplementary/netlink
sudo ./log_to_file ~/csi.dat
```

In another terminal, connect the interface to an **open** 2.4 GHz **802.11n HT** access point, then generate traffic:

```bash
ping -I ${IFACE} <AP_IP_ADDRESS>
```

Why

The CSI tool FAQ says CSI is measured only on **received 802.11n HT packets** under compatible physical/logical settings. It also says encrypted WEP/WPA/WPA2 networks are a problem because the modified firmware does not have enough code room for both CSI paths and encryption paths. Dhalperi

Cross-check

In a third terminal:

```bash
watch -n 1 ls -lh ~/csi.dat
```

If the file keeps growing, your Intel 5300 CSI path is alive.

Very important rule after installation

From now on, always check:

```bash
uname -r
uname -r | cut -d. -f1-2
```

If the second command is not `4.15`, do **not** continue with rebuilding or troubleshooting the CSI driver on that kernel. The spanev instructions are explicit that the kernel should be **4.15**. GitHub

Also:

- do **not** run `do-release-upgrade`
- do **not** move this collector machine to a newer Ubuntu release
- do **not** intentionally install HWE kernel meta-packages

That is the equivalent of changing the engine in the middle of the experiment.

What I recommend next, immediately after this works

Once `~/csi.dat` is growing, do **not** collect your full dataset yet. First do these three things:

1. move from the simple AP ping test to **monitor-mode collection**, because the CSI tool authors say they found the tool most effective in **monitor mode with injection scripts**, and that client/AP/ad-hoc modes were less reliable,
2. add a thin **GUI wrapper** around setup/start/stop/session metadata,
3. convert every raw `csi.dat` into **HDF5** immediately after collection for long-term research use. Dhalperi

The next thing I should give you is the **post-install phase**:

- exact **monitor-mode setup** on Ubuntu 18.04,
- the **GUI collection wrapper design**,
- and the **raw CSI → HDF5 conversion pipeline**.

Sources

go for each setp next

I'm moving from "installation works" to the part that actually turns it into a research tool: monitor-mode collection, a GUI-controlled session flow, and a future-proof HDF5 pipeline. I'm verifying the exact helper scripts and parser options before I lay out the commands so you can follow them without guesswork.

I have the critical part confirmed: for serious experiments, the original authors explicitly recommend **monitor-mode injection**, not normal AP/client mode, and the injection README gives the exact receiver/transmitter flow. I'm structuring the rest around that so you don't build your dataset on the weakest operating mode.

Thought for 3m 49s ›

Perfect. We'll now move from **"the driver works"** to **"the lab pipeline works."**

Big picture first: your earlier smoke test was like checking whether a camera can turn on. What we do now is set up the **actual filming workflow**: controlled packets, monitor-mode capture, session metadata, GUI control, and canonical HDF5 storage. For the Intel 5300 CSI Tool, this is the right direction because the original FAQ says the tool was most effective in **monitor mode with injection scripts**, and the injection README explicitly separates a **receiver** flow and a **transmitter** flow. The same FAQ also says CSI is only measured on compatible **802.11n HT** packets, and in monitor mode it is tied to the script's fixed addressing pattern rather than arbitrary overheard traffic. Dhalperi +2

I'll assume **Steps 1–12 already work** on your Ubuntu 18.04 / kernel 4.15 collector.

Phase A — Move to real monitor-mode collection

Step 13 — Understand the topology before running commands

For **proper controlled CSI collection**, use **two devices**:

- **Receiver** = your Intel 5300 Ubuntu 18.04 collector
- **Transmitter** = a second Linux machine that sends controlled packets

Why? Because the supplementary injection README literally uses a **receiver** path and a **transmitter** path, and the transmitter side relies on the injection utilities rather than normal Wi-Fi traffic. It also notes that you need **LORCON v1** plus `libpcap-dev` to compile the injection executable. GitHub

Think of it like this:

- the **receiver** is the microphone,
- the **transmitter** is the person speaking scripted sentences.

If you let the environment "speak randomly," your CSI becomes messy and harder to reproduce.

Step 14 — Receiver preflight checks

Run on the **Intel 5300 collector**:

```bash
export IFACE=$(iw dev | awk '$1=="Interface"{print $2; exit}')
echo "$IFACE"

mount | grep debugfs || sudo mount -t debugfs none /sys/kernel/debug

uname -r
iw dev
ip link show "$IFACE"
```

Why we do this

- `IFACE=...` automatically discovers your wireless interface name.
- `debugfs` must be mounted because rate control in monitor/injection mode is set through debugfs.
- `uname -r` confirms you are still on the right kernel.
- `iw dev` and `ip link` confirm the interface exists and is up/down in a controllable state.

Cross-check

You want:

- a valid interface name printed, like `wlp3s0`
- `debugfs` mounted
- kernel still showing `4.15.x`

If `IFACE` comes out empty, stop and fix the wireless driver first.

Step 15 — Pick a clean channel and start with HT20

Before monitor mode, do a quick scan:

```bash
sudo iw dev "$IFACE" scan | egrep 'SSID|primary channel|signal'
```

Why we do this

You want a **quiet channel** for controlled experiments. Start with **HT20**, not HT40. HT20 is the safer lab baseline.

My recommendation

- Start with **channel 6 HT20** on 2.4 GHz for bring-up
- Later move to a quieter fixed channel after surveying your environment

Cross-check

Choose a channel where:

- there are fewer nearby APs
- RSSI from other networks is weaker
- the environment is stable during your collection window

Step 16 — Build the transmitter-side injection tools

Run these on the **transmitter machine**, not the receiver.

Install prerequisites:

```bash
sudo apt update
sudo apt install build-essential libpcap-dev git
```

Clone and build LORCON v1:

```bash
cd ~
git clone https://github.com/dhalperi/lorcon-old.git
cd lorcon-old
./configure
make
sudo make install
sudo ldconfig
```

Build the CSI supplementary injection tools:

```bash
```

```
cd ~
git clone https://github.com/dhalperi/linux-80211n-csitool-supplementary.g
cd linux-80211n-csitool-supplementary/injection
make
```

Why we do this

The injection README says you need **LORCON version 1** and `libpcap-dev` to compile the injection executable, then `make install here` / `make` in the injection directory. That is what enables the scripted transmitter side. GitHub +1

Cross-check

```bash
ls -l ~/linux-80211n-csitool-supplementary/injection/random_packets
```

You want the `random_packets` executable to exist.

Step 17 — Put the receiver into monitor mode

Run on the **Intel 5300 receiver**:

```bash
cd ~/linux-80211n-csitool-supplementary/injection

grep -n "wlan0" setup_monitor_csi.sh
```

If your interface is **not** `wlan0`, edit the script:

```bash
sed -i "s/wlan0/$IFACE/g" setup_monitor_csi.sh
```

Now run monitor setup:

```bash
sudo ./setup_monitor_csi.sh 6 HT20
```

Why we do this

The injection README shows the receiver flow as:

```bash
./setup_monitor_csi.sh 64 HT20
sudo ../netlink/log_to_file log.dat
```

So `setup_monitor_csi.sh` is the intended receiver-side entry point for monitor-mode CSI capture. We are just using channel `6` instead of `64` for a simpler first lab setup.

GitHub

Cross-check

```bash
iw dev "$IFACE" info
iwconfig "$IFACE"
```

You want to see:

- interface type = **monitor**
- channel set to your chosen channel
- no obvious errors in the script output

Step 18 — Lock the monitor-mode injection rate

On the **receiver**, set the monitor injection rate path:

```bash
echo 0x1c113 | sudo tee /sys/kernel/debug/ieee80211/phy0/iwlwifi/iwldvm/de
```

Why we do this

The CSI Tool FAQ says the monitor-mode transmit rate is controlled through this exact debugfs entry. Fixed-rate experiments are easier to reproduce than adaptive-rate experiments. Dhalperi

Cross-check

```bash
cat /sys/kernel/debug/ieee80211/phy0/iwlwifi/iwldvm/debug/monitor_tx_rate
```

It should print the value you wrote.

Step 19 — Start CSI logging on the receiver

Run on the **receiver**:

```bash
mkdir -p ~/csi_sessions/test01
cd ~/linux-80211n-csitool-supplementary/netlink
sudo ./log_to_file ~/csi_sessions/test01/session_raw.csi.dat
```

Why we do this

`log_to_file` is the userspace logger that records the CSI notifications into a raw `.dat` trace. This is your **raw ground-truth archive**. GitHub +1

Cross-check

In another terminal:

```bash
watch -n 1 ls -lh ~/csi_sessions/test01/session_raw.csi.dat
```

It should grow once packets start arriving.

Step 20 — Put the transmitter into inject mode

On the **transmitter**:

```bash
export TX_IFACE=$(iw dev | awk '$1=="Interface"{print $2; exit}')
echo "$TX_IFACE"

cd ~/linux-80211n-csitool-supplementary/injection
```

```
grep -n "wlan0" setup_inject.sh
sed -i "s/wlan0/$TX_IFACE/g" setup_inject.sh

sudo ./setup_inject.sh 6 HT20
```

Why we do this

The injection README shows the transmitter flow as:

```bash
./setup_inject.sh 64 HT20
echo 0x4101 | sudo tee `find /sys -name monitor_tx_rate`
sudo ./random_packets 1 100 1
```

So this is the intended controlled-transmission path. GitHub

Cross-check

```bash
iw dev "$TX_IFACE" info
iwconfig "$TX_IFACE"
```

You want monitor/injection state configured.

Step 21 — Fix the transmitter rate and send controlled packets

On the **transmitter**:

```bash
echo 0x4101 | sudo tee "$(find /sys -name monitor_tx_rate | head -n 1)"
sudo ./random_packets 1 100 1
```

Why we do this

The README uses exactly this pattern for transmitter-side injection. The important point is not the magic number itself at first; it is that you are using the **same controlled script path** each time so your trials are reproducible. In monitor mode, the FAQ says CSI is collected only for packets using the expected addressing/conditions, which is why using the provided injection workflow matters. GitHub +1

Cross-check

Back on the **receiver**, your `session_raw.csi.dat` file should now grow steadily.

Step 22 — Stop the capture cleanly

On the **receiver**, stop `log_to_file` with `Ctrl+C`.

Now verify:

```bash
ls -lh ~/csi_sessions/test01/session_raw.csi.dat
```

Why we do this

Each collection session should produce **one raw file**, not one giant mixed log. This makes later labeling and paper reproducibility much easier.

Step 23 — Optional: log payloads or sequence numbers too

If later you want packet payload / sequence-number alignment, reload the driver with a different mask:

```bash
sudo modprobe -r iwldvm iwlwifi mac80211 || sudo modprobe -r iwlwifi mac80
sudo modprobe iwlwifi connector_log=0x5
```

Why we do this

The FAQ says:

- `0x1` logs beamforming information only
- `0x5` combines CSI with packet payload logging (`IWL_CONN_BFEE_NOTIF_MSK | IWL_CONN_RX_MPDU_MSK`) Dhalperi

This is useful later if you want tighter packet-level synchronization.

Phase B — Give yourself a GUI on Ubuntu 18.04

Because your collector OS is old, the cleanest GUI is a **thin wrapper**, not a giant custom app. The GUI should be the **dashboard**, while the driver/scripts remain the **engine**.

Step 24 — Install a simple GUI toolkit

On the **receiver**:

```bash
sudo apt install zenity
mkdir -p ~/bin
```

Why we do this

`zenity` gives you a lightweight GNOME GUI form without dragging in a full framework. On an older collector OS, that is the most robust choice.

Step 25 — Create a GUI-based collection launcher

Create this file:

```bash
nano ~/bin/csi_collect_gui.sh
```

Paste this:

```bash
#!/usr/bin/env bash
set -euo pipefail

BASE_DIR="$HOME/csi_sessions"
INJ_DIR="$HOME/linux-80211n-csitool-supplementary/injection"
LOG_BIN="$HOME/linux-80211n-csitool-supplementary/netlink/log_to_file"

IFACE=$(iw dev | awk '$1=="Interface"{print $2; exit}')
[ -n "${IFACE:-}" ] || { zenity --error --text="No wireless interface foun

FORM=$(zenity --forms \
  --title="Intel 5300 CSI Collector" \
  --text="Enter session parameters" \
```

```
  --add-entry="Subject ID" \
  --add-entry="Session ID" \
  --add-entry="Trial ID" \
  --add-entry="Room ID" \
  --add-entry="Path ID" \
  --add-entry="Direction (L2R/R2L/FWD/BWD)" \
  --add-entry="Channel" \
  --add-entry="Bandwidth (HT20/HT40)" \
  --add-entry="Duration seconds" \
  --add-entry="Notes")

[ $? -eq 0 ] || exit 0

SUBJECT=$(echo "$FORM" | cut -d'|' -f1)
SESSION=$(echo "$FORM" | cut -d'|' -f2)
TRIAL=$(echo "$FORM" | cut -d'|' -f3)
ROOM=$(echo "$FORM" | cut -d'|' -f4)
PATH_ID=$(echo "$FORM" | cut -d'|' -f5)
DIRECTION=$(echo "$FORM" | cut -d'|' -f6)
CHANNEL=$(echo "$FORM" | cut -d'|' -f7)
BW=$(echo "$FORM" | cut -d'|' -f8)
DURATION=$(echo "$FORM" | cut -d'|' -f9)
NOTES=$(echo "$FORM" | cut -d'|' -f10)

STAMP=$(date +"%Y%m%d_%H%M%S")
OUTDIR="$BASE_DIR/subj_${SUBJECT}/sess_${SESSION}/trial_${TRIAL}_${STAMP}"
mkdir -p "$OUTDIR"

cat > "$OUTDIR/session.json" <<EOF
{
  "subject_id": "${SUBJECT}",
  "session_id": "${SESSION}",
  "trial_id": "${TRIAL}",
  "room_id": "${ROOM}",
  "path_id": "${PATH_ID}",
  "direction": "${DIRECTION}",
  "channel": "${CHANNEL}",
  "bandwidth": "${BW}",
  "duration_s": "${DURATION}",
  "interface": "${IFACE}",
  "notes": "${NOTES}",
  "start_time": "$(date --iso-8601=seconds)"
}
EOF

cd "$INJ_DIR"

if grep -q "wlan0" setup_monitor_csi.sh; then
  sed -i "s/wlan0/$IFACE/g" setup_monitor_csi.sh
```

```
fi

sudo ./setup_monitor_csi.sh "$CHANNEL" "$BW"

sudo "$LOG_BIN" "$OUTDIR/session_raw.csi.dat" &
LOGGER_PID=$!

sleep 2
zenity --info --text="Logging started.\n\nOutput: $OUTDIR\n\nClick OK to b

sleep "$DURATION"

sudo kill "$LOGGER_PID" || true
wait "$LOGGER_PID" 2>/dev/null || true

cat > "$OUTDIR/end.json" <<EOF
{
  "end_time": "$(date --iso-8601=seconds)"
}
EOF

zenity --info --text="Collection complete.\n\nSaved in:\n$OUTDIR"
```

Make it executable:

```bash
chmod +x ~/bin/csi_collect_gui.sh
```

Run it:

```bash
~/bin/csi_collect_gui.sh
```

Why this script is good

It does the 4 jobs you actually need:

1. asks for session parameters using a GUI
2. creates a clean folder structure
3. writes session metadata to JSON
4. runs monitor-mode setup and `log_to_file` automatically

That means your dataset becomes **organized at the moment of capture**, not weeks later when you are trying to remember what “test3_final_reallyfinal.dat” meant.

Cross-check

After one run, verify:

```bash
find ~/csi_sessions -maxdepth 4 -type f | sort
```

You want to see:

- `session_raw.csi.dat`
- `session.json`
- `end.json`

Phase C — Convert raw `.dat` into canonical HDF5

CSIKit explicitly supports **Intel IWL5300**, provides both CLI and library parsing, and can convert CSI data into other serializations. It also documents that each `CSIFrame` contains a `csi_matrix`, while the IWL5300 frame type includes fields such as `timestamp_low`, `bfee_count`, `n_rx`, `n_tx`, `rssi_a`, `rssi_b`, `rssi_c`, `noise`, `agc`, `antenna_sel`, `length`, and `rate`. gi-z.github.io +3

HDF5 is the right canonical format here because h5py datasets support **chunking, compression, checksums, resizable shapes, and descriptive attributes**. That is exactly what you want for packet-level CSI plus experiment metadata. docs.h5py.org +2

Step 26 — Prepare the modern analysis environment

Do this on your **modern research machine**, not necessarily the old collector:

```bash
sudo apt update
sudo apt install python3 python3-venv python3-pip
python3 -m venv ~/venvs/csi-research
source ~/venvs/csi-research/bin/activate
pip install numpy h5py csikit
```

Why we do this

This keeps all parsing and conversion work on a modern, clean Python environment.

Cross-check

Bash

```bash
python - <<'PY'
import numpy, h5py, CSIKit
print("OK")
PY
```

Step 27 — Create the raw-to-HDF5 converter

Create a file:

Bash

```bash
nano ~/dat_to_h5.py
```

Paste this:

Python Run

```python
#!/usr/bin/env python3
import json
import os
import sys
import numpy as np
import h5py

from CSIKit.reader import IWLBeamformReader

def safe_attr(frame, name, default=0):
    return getattr(frame, name, default)

def main(dat_path, meta_path, out_path):
    with open(meta_path, "r", encoding="utf-8") as f:
        meta = json.load(f)

    reader = IWLBeamformReader()
    csi_data = reader.read_file(dat_path, scaled=True)

    frames = csi_data.frames
    if not frames:
        raise RuntimeError("No CSI frames found.")
```

```
    first = np.asarray(frames[0].csi_matrix)
    if first.ndim != 3:
        raise RuntimeError(f"Unexpected CSI matrix shape: {first.shape}")

    n_frames = len(frames)
    n_sub, n_rx, n_tx = first.shape

    csi = np.zeros((n_frames, n_sub, n_rx, n_tx), dtype=np.complex64)
    timestamp_low = np.zeros(n_frames, dtype=np.uint32)
    bfee_count = np.zeros(n_frames, dtype=np.uint16)
    rssi_a = np.zeros(n_frames, dtype=np.int16)
    rssi_b = np.zeros(n_frames, dtype=np.int16)
    rssi_c = np.zeros(n_frames, dtype=np.int16)
    noise = np.zeros(n_frames, dtype=np.int16)
    agc = np.zeros(n_frames, dtype=np.int16)
    antenna_sel = np.zeros(n_frames, dtype=np.uint8)
    length = np.zeros(n_frames, dtype=np.uint16)
    rate = np.zeros(n_frames, dtype=np.uint16)

    for i, frame in enumerate(frames):
        csi[i] = np.asarray(frame.csi_matrix, dtype=np.complex64)
        timestamp_low[i] = safe_attr(frame, "timestamp_low", i)
        bfee_count[i] = safe_attr(frame, "bfee_count", i)
        rssi_a[i] = safe_attr(frame, "rssi_a", 0)
        rssi_b[i] = safe_attr(frame, "rssi_b", 0)
        rssi_c[i] = safe_attr(frame, "rssi_c", 0)
        noise[i] = safe_attr(frame, "noise", 0)
        agc[i] = safe_attr(frame, "agc", 0)
        antenna_sel[i] = safe_attr(frame, "antenna_sel", 0)
        length[i] = safe_attr(frame, "length", 0)
        rate[i] = safe_attr(frame, "rate", 0)

    amplitude = np.abs(csi).astype(np.float32)
    phase = np.angle(csi).astype(np.float32)

    os.makedirs(os.path.dirname(out_path), exist_ok=True)

    with h5py.File(out_path, "w") as f:
        g_session = f.create_group("session")
        g_csi = g_session.create_group("csi")
        g_packet = g_session.create_group("packet")
        g_labels = g_session.create_group("labels")
        g_meta = g_session.create_group("meta")

        chunk_frames = min(256, n_frames)

        g_csi.create_dataset(
            "data",
            data=csi,
```

```
        chunks=(chunk_frames, n_sub, n_rx, n_tx),
        compression="gzip",
        compression_opts=4,
        shuffle=True,
        fletcher32=True
    )

    g_csi.create_dataset(
        "amplitude",
        data=amplitude,
        chunks=(chunk_frames, n_sub, n_rx, n_tx),
        compression="gzip",
        compression_opts=4,
        shuffle=True,
        fletcher32=True
    )

    g_csi.create_dataset(
        "phase",
        data=phase,
        chunks=(chunk_frames, n_sub, n_rx, n_tx),
        compression="gzip",
        compression_opts=4,
        shuffle=True,
        fletcher32=True
    )

    g_packet.create_dataset("timestamp_low", data=timestamp_low)
    g_packet.create_dataset("bfee_count", data=bfee_count)
    g_packet.create_dataset("rssi_a", data=rssi_a)
    g_packet.create_dataset("rssi_b", data=rssi_b)
    g_packet.create_dataset("rssi_c", data=rssi_c)
    g_packet.create_dataset("noise", data=noise)
    g_packet.create_dataset("agc", data=agc)
    g_packet.create_dataset("antenna_sel", data=antenna_sel)
    g_packet.create_dataset("length", data=length)
    g_packet.create_dataset("rate", data=rate)

    # session labels: keep experiment annotations separate from packet
    for k in ["subject_id", "session_id", "trial_id", "room_id", "path
        if k in meta:
            g_labels.attrs[k] = str(meta[k])

    # metadata: acquisition context
    for k, v in meta.items():
        g_meta.attrs[k] = str(v)

    g_meta.attrs["source_dat"] = os.path.abspath(dat_path)
    g_meta.attrs["scaled"] = "True"
```

```
        g_meta.attrs["n_frames"] = str(n_frames)
        g_meta.attrs["n_subcarriers"] = str(n_sub)
        g_meta.attrs["n_rx"] = str(n_rx)
        g_meta.attrs["n_tx"] = str(n_tx)

    print(f"Wrote {out_path}")
    print(f"Frames={n_frames}, subcarriers={n_sub}, rx={n_rx}, tx={n_tx}")

if __name__ == "__main__":
    if len(sys.argv) != 4:
        print("Usage: dat_to_h5.py <session_raw.csi.dat> <session.json> <o
        sys.exit(1)
    main(sys.argv[1], sys.argv[2], sys.argv[3])
```

Make it executable:

```bash
chmod +x ~/dat_to_h5.py
```

Why this script is structured this way

- `csi/data` stores the **raw complex CSI**
- `csi/amplitude` and `csi/phase` store derived views you will use constantly
- `packet/*` stores packet-level metadata from IWL5300 frames
- `labels` keeps human experiment labels separate
- `meta` stores acquisition context from your GUI JSON

This is the clean scientific-file equivalent of putting your lab notebook, sensor readings, and experiment labels in one place.

Step 28 — Convert one session

Run:

```bash
```

```
source ~/venvs/csi-research/bin/activate

python ~/dat_to_h5.py \
  ~/csi_sessions/subj_001/sess_001/trial_001_20260319_120000/session_raw.c
  ~/csi_sessions/subj_001/sess_001/trial_001_20260319_120000/session.json
  ~/canonical/subj_001_sess_001_trial_001.h5
```

Cross-check

‹/› Bash

```
python - <<'PY'
import h5py
f = h5py.File("/home/$USER/canonical/subj_001_sess_001_trial_001.h5".repla
print(list(f["session"].keys()))
print(f["session/csi/data"].shape)
print(dict(f["session/labels"].attrs))
print(dict(f["session/meta"].attrs))
f.close()
PY
```

You want to see:

- groups: `csi` , `packet` , `labels` , `meta`
- a sensible CSI shape
- your subject/session labels present

Phase D — What to do immediately after this

Once the above works, your workflow becomes:

1. open GUI
2. enter subject/session/trial parameters
3. receiver enters monitor mode
4. transmitter injects controlled HT packets
5. raw `.dat` gets saved
6. metadata JSON gets saved
7. convert to `.h5`
8. only then start preprocessing / segmentation / model training

That gives you a **journal-grade data foundation** instead of a pile of anonymous logs.

The most important technical truth to remember is this: with the Intel 5300 tool, **ambient Wi-Fi is not your friend**. Controlled monitor-mode collection is the difference between noisy hobbyist data and publishable research data. The FAQ is very clear on that.

Dhalperi +1

The next best continuation is the **preprocessing stage**: packet cleaning, antenna selection, amplitude/phase handling, denoising, gait-window segmentation, and the first baseline model.

Sources

create me a prompt that I want to run in gpt's deep research mode to perform this task. If it need to read any research paper then make sure it should read during the response.

Thought for 25s

Here's a strong **copy-paste prompt** for GPT's **Deep Research** mode:

```
I want you to act as a senior WiFi-CSI research engineer and research

- Processor: Intel i7 10th gen
- NIC card: Intel 5300
- SSD: 256 GB

Your job is to produce a complete, first-to-last, deeply researched g

Important instructions:
1. You must actively browse the web and read relevant official docume
2. If any important claim depends on a paper, open the paper or its o
3. Do not just list paper titles. Read enough of the actual paper con
4. Use citations throughout the answer for all major factual claims.
5. If something is uncertain, outdated, or version-sensitive, verify
6. Do not give a shallow summary. I want a detailed, practical, techn
7. Do not ask me follow-up questions unless absolutely necessary. Mak

I want the final answer to be organized into the following sections:

SECTION A — Executive recommendation
- Tell me whether Intel 5300 is still a good choice in 2026 for WiFi-
- Compare Intel 5300 with newer CSI-capable platforms if relevant.
- Tell me whether I should use my main machine or a separate collecto
```

```
- Clearly state the best Ubuntu version for the Intel 5300 collection
- Clearly state whether the latest Ubuntu will work or not, and why.

SECTION B — Full environment setup for Intel 5300 CSI data collection
I need a step-by-step setup guide from zero to working CSI collection

For every step:
- Give the exact Ubuntu commands.
- Explain why each command is needed.
- Give cross-check / verification commands after each step so I can c
- Tell me the expected output or expected state.
- Warn me about common failure points and how to debug them.

This setup section must include:
- OS version choice
- kernel version constraints
- driver setup
- firmware replacement if needed
- module loading
- monitor mode / injection mode if relevant
- logging pipeline
- how to test whether CSI is actually being captured
- how to avoid breaking the working setup later

SECTION C — Practical collection architecture
Design the actual collection system for a publishable gait dataset.

Include:
- whether Intel 5300 should be receiver only
- whether I need a separate transmitter
- recommended network topology
- AP/client mode vs monitor/injection mode
- controlled experiment design for gait collection
- room setup
- LOS/NLOS conditions
- channel and bandwidth choices
- repeatability strategy
- session metadata to record

SECTION D — GUI requirement
I need a GUI so I can control collection parameters during data colle

Please propose the best practical GUI architecture for this Intel 530
- whether to build a thin wrapper over the CLI tools
- what parameters the GUI should expose
- how the GUI should start/stop collection
- how the GUI should write session metadata
- whether PyQt / Zenity / another lightweight option is best
- a recommended folder structure for saved sessions
```

```
If useful, include a minimal but practical implementation plan.

SECTION E — Best data format for future research
I want the collected data stored in the best format for future resear

You must:
- compare raw .dat, CSV, HDF5, Parquet, and any other relevant format
- recommend the best archival format and explain why
- recommend the best format for derived features
- propose a full HDF5 schema for WiFi CSI gait data
- explain how to preserve raw data, metadata, labels, and experiment
- explain how to support future preprocessing and model training

SECTION F — Research-paper-grounded pipeline for a Q1-level study
This is very important.

Read relevant recent papers on:
- WiFi CSI gait recognition
- WiFi sensing for gait / person identification
- cross-domain / cross-environment WiFi sensing
- open-set gait recognition if relevant
- domain generalization / domain adaptation for WiFi CSI
- benchmark datasets such as Widar / SenseFi / other relevant dataset

Then produce a research-grounded pipeline that includes:
- dataset design
- preprocessing
- segmentation
- feature representations
- baseline models
- strong modern models
- evaluation metrics
- train/validation/test protocols
- cross-room / cross-day / cross-subject testing
- open-set evaluation if appropriate
- ablation studies
- reproducibility checklist

For each major recommendation, tie it back to current literature and

SECTION G — Best publishable research direction
After reading the literature, propose 3 strong paper ideas for me.

For each idea include:
- novelty
- difficulty
- data requirements
- expected contribution
```

```
- key baselines
- risks
- target Q1 journals
- why it could be publishable

Then choose the single best one for my hardware and situation.

SECTION H — Q1 journal targeting
Recommend the best realistic Q1 journals for this project.
For each journal:
- explain the fit
- explain what kind of contribution it expects
- tell me what would make my work too weak for that journal
- tell me what would strengthen the paper

SECTION I — Final actionable roadmap
End with a practical roadmap in phases:
Phase 1: environment setup
Phase 2: first CSI capture
Phase 3: stable controlled collection
Phase 4: GUI and data management
Phase 5: dataset collection
Phase 6: preprocessing and baselines
Phase 7: advanced model
Phase 8: paper writing and submission

For each phase:
- objectives
- deliverables
- pitfalls
- success criteria

Additional constraints:
- Prefer official documentation and primary sources when possible.
- When using research papers, prefer recent and high-quality papers,
- If a tool, repo, or setup is outdated, say so clearly.
- If my 10th gen Intel laptop may have compatibility issues with old
- Be honest when something is fragile or legacy.
- I care more about correctness and reproducibility than convenience.
- Make the answer detailed and technically rigorous.
```

A slightly stronger variation is to add this line near the top:

```
nd recent WiFi-CSI gait papers, then use those findings as the basis of
```

The best way to use this is to paste it into Deep Research exactly as-is.